EXP 25 Write the python program to implement Feed forward neural Network.

AIM:

To write and implement a Python program to create a Feed Forward Neural Network (FFNN) using TensorFlow and Keras to train the model and predict the output based on the given input data.

Program:

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import numpy as np

# Input data (XOR problem)
X = np.array([[0,0], [0,1], [1,0], [1,1]])
Y = np.array([[0], [1], [1], [0]])

# Create Feed Forward Neural Network
model = keras.Sequential([
    layers.Dense(4, activation='relu', input_shape=(2,)),
    layers.Dense(1, activation='sigmoid')
])

# Compile the model
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# Train the model
model.fit(X, Y, epochs=1000, verbose=0)

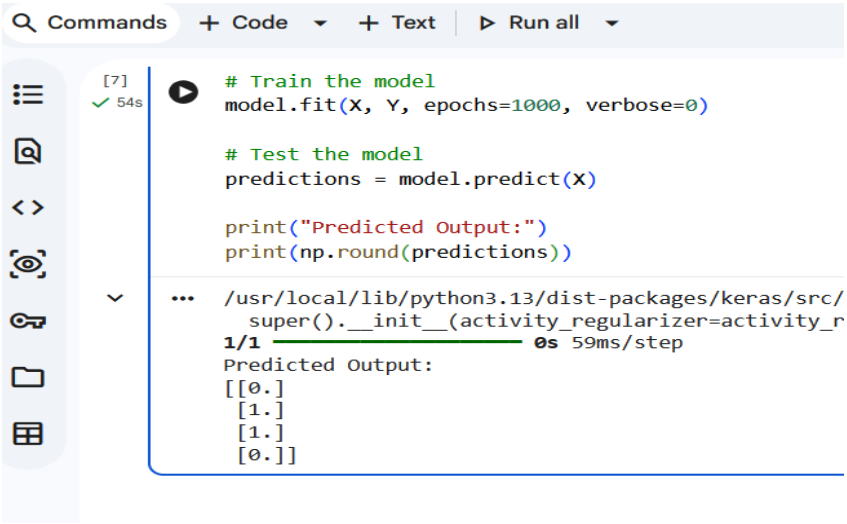
# Test the model
```

predictions = model.predict(X)

print("Predicted Output:")

print(np.round(predictions))

Output:



EXP 26 Write a Prolog Program to Sum the Integers from 1 to n.

AIM:

To write and implement a Prolog program to calculate the sum of integers from 1 to N using recursion.

Program:

prolog_code = """

sum(0, 0).

sum(N, S) :-

N > 0,

N1 is N - 1,

sum(N1, S1),

S is N + S1.

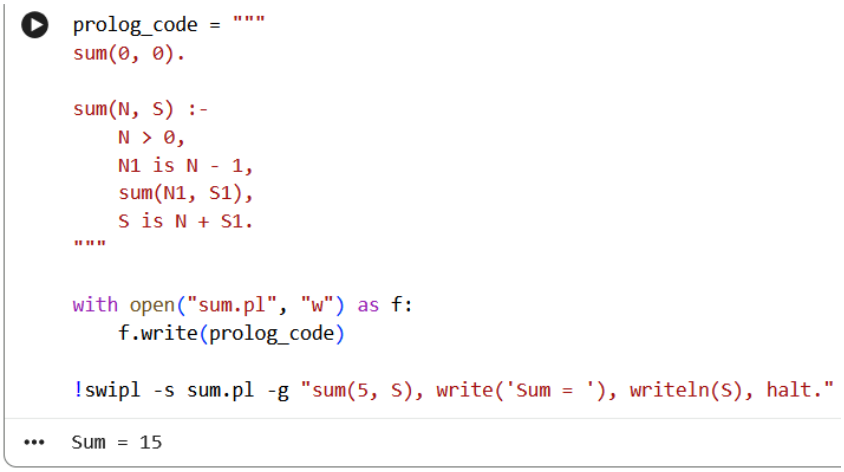
"""

with open("sum.pl", "w") as f:

f.write(prolog_code)

!swipl -s sum.pl -g "sum(5, S), write('Sum = '), writeln(S), halt."

Output:



EXP 27 Write a Prolog Program for A DB WITH NAME, DOB.

AIM

To write and implement a Prolog program to create a database containing Name and Date of Birth (DOB) and retrieve and display the stored records.

Program:

prolog_code = """

person('Arun', '2002-05-10').

person('Priya', '2003-08-15').

person('Kumar', '2001-12-20').

person('Divya', '2002-03-25').

display :-

person(Name, DOB),

write('Name: '), write(Name),

write(', DOB: '), write(DOB), nl,

fail.

display.

"""

with open("database.pl", "w") as f:

  f.write(prolog_code)

!swipl -s database.pl -g "display, halt."

Output:

```
    display.
    """

    with open("database.pl", "w") as f:
        f.write(prolog_code)

    !swipl -s database.pl -g "display, halt."
```

```
...  Name: Arun, DOB: 2002-05-10
    Name: Priya, DOB: 2003-08-15
    Name: Kumar, DOB: 2001-12-20
    Name: Divya, DOB: 2002-03-25
```

EXP 28 : Write a Prolog Program for STUDENT TEACHER-SUB-CODE.

AIM

To write and implement a Prolog program to create a database containing Student Name, Teacher Name, and Subject Code and retrieve and display the stored records.

Program:

prolog_code = """

student_teacher('Arun', 'Ramesh', 'CS101').

student_teacher('Priya', 'Kumar', 'CS102').

student_teacher('Divya', 'Suresh', 'CS103').

student_teacher('Rahul', 'Anitha', 'CS104').

display :-

  student_teacher(Student, Teacher, SubCode),

  write('Student: '), write(Student),

  write(', Teacher: '), write(Teacher),

```
    write(', Subject Code: '), write(SubCode), nl,
    fail.

display.
"""

with open("student.pl", "w") as f:
    f.write(prolog_code)

!swipl -s student.pl -g "display, halt."
```

Output:

```
        write('Student: '), write(Student),
        write(', Teacher: '), write(Teacher),
        write(', Subject Code: '), write(SubCode), nl,
        fail.

    display.
    """

    with open("student.pl", "w") as f:
        f.write(prolog_code)

    !swipl -s student.pl -g "display, halt."
```

```
Student: Arun, Teacher: Ramesh, Subject Code: CS101
Student: Priya, Teacher: Kumar, Subject Code: CS102
Student: Divya, Teacher: Suresh, Subject Code: CS103
Student: Rahul, Teacher: Anitha, Subject Code: CS104
```

EXP 29: Write a Prolog Program for PLANETS DB.

AIM

To write and implement a Prolog program to create a Planet Database (PLANETS DB) and retrieve and display the details of different planets.

Program:

```
prolog_code = """

planet(mercury, 4879, terrestrial).

planet(venus, 12104, terrestrial).
```

```
planet(earth, 12742, terrestrial).
planet(mars, 6779, terrestrial).
planet(jupiter, 139820, gas_giant).
planet(saturn, 116460, gas_giant).
planet(uranus, 50724, ice_giant).
planet(neptune, 49244, ice_giant).

display :-
    planet(Name, Diameter, Type),
    write('Planet: '), write(Name),
    write(', Diameter (km): '), write(Diameter),
    write(', Type: '), write(Type), nl,
    fail.

display.
"""

with open("planets.pl", "w") as f:
    f.write(prolog_code)

!swipl -s planets.pl -g "display, halt."
```

Output:

```
display.
"""

with open("planets.pl", "w") as f:
    f.write(prolog_code)

!swipl -s planets.pl -g "display, halt."
```

```
Planet: mercury, Diameter (km): 4879, Type: terrestrial
Planet: venus, Diameter (km): 12104, Type: terrestrial
Planet: earth, Diameter (km): 12742, Type: terrestrial
Planet: mars, Diameter (km): 6779, Type: terrestrial
Planet: jupiter, Diameter (km): 139820, Type: gas_giant
Planet: saturn, Diameter (km): 116460, Type: gas_giant
Planet: uranus, Diameter (km): 50724, Type: ice_giant
Planet: neptune, Diameter (km): 49244, Type: ice_giant
```

Exp 30 Write a Prolog Program to implement Towers of Hanoi.

Aim:

To write and implement a Prolog program to solve the Towers of Hanoi problem using recursion.

Program:

```
prolog_code = """
hanoi(1, Source, Destination, _) :-
   write('Move disk from '),
   write(Source),
   write(' to '),
   write(Destination), nl.


hanoi(N, Source, Destination, Auxiliary) :-
   N > 1,
   N1 is N - 1,
   hanoi(N1, Source, Auxiliary, Destination),
   hanoi(1, Source, Destination, Auxiliary),
   hanoi(N1, Auxiliary, Destination, Source).
"""


with open("hanoi.pl", "w") as f:
   f.write(prolog_code)


!swipl -s hanoi.pl -g "hanoi(3, a, c, b), halt."
```

Output:

```
        hanoi(N1, Auxiliary, Destination, Source).
    """

    with open("hanoi.pl", "w") as f:
        f.write(prolog_code)

    !swipl -s hanoi.pl -g "hanoi(3, a, c, b), halt."
```

```
Move disk from a to c
Move disk from a to b
Move disk from c to b
Move disk from a to c
Move disk from b to a
Move disk from b to c
Move disk from a to c
```